

1. Introducción

La evolución de la ingeniería de software ha impulsado el uso de enfoques que reduzcan la separación existente entre la especificación de requisitos, el diseño del sistema y su implementación. Cuando estas etapas se desarrollan como actividades independientes, los cambios realizados en los requisitos o en los modelos pueden no reflejarse correctamente en el código fuente, generando inconsistencias, retrabajo y defectos. En este contexto, la Ingeniería Dirigida por Modelos o Model-Driven Engineering (MDE) considera los modelos como artefactos centrales del proceso de desarrollo, mientras que el Model-Driven Development (MDD) aplica este principio para obtener, total o parcialmente, artefactos de implementación mediante transformaciones automatizadas. La arquitectura MDA organiza este proceso mediante los niveles CIM, PIM y PSM, permitiendo avanzar desde la representación del dominio hasta modelos vinculados con una plataforma tecnológica específica[1],[2].

La aplicación de este enfoque requiere que los requisitos sean claros, verificables y trazables. La norma ISO/IEC/IEEE 29148:2018 establece principios para documentar y gestionar requisitos a lo largo del ciclo de vida del software, de manera que puedan relacionarse con los modelos de diseño, los componentes implementados y sus respectivas pruebas. De forma complementaria, UML proporciona una notación normalizada para representar la estructura y el comportamiento del sistema mediante clases, asociaciones, casos de uso, secuencias y otros elementos[3]. En un proceso MDD, estos modelos dejan de ser únicamente representaciones gráficas y se convierten en entradas procesables para transformaciones modelo-a-modelo y modelo-a-texto, estas últimas orientadas a producir código fuente, configuraciones o documentación[4],[3].

Esta relación resulta pertinente para el proyecto MundiPets, concebido como una plataforma web orientada a facilitar procesos de adopción y cruce responsable de mascotas. Su especificación de requisitos de software contempla la gestión de perfiles de mascotas, historiales médicos, documentos veterinarios, publicaciones, solicitudes de adopción o cruce, comunicación entre usuarios, privacidad de la información, validación profesional y componentes de inteligencia artificial. Asimismo, el proyecto identifica actores como propietarios, adoptantes, interesados en cruce, refugios y veterinarias, junto con requisitos de seguridad, disponibilidad, usabilidad, integridad y arquitectura modular. Estos elementos ofrecen una base apropiada para construir un modelo de clases compuesto por entidades como "Usuario", "Mascota", "HistorialMedico", "DocumentoVeterinario", "Publicacion", "Solicitud", "Mensaje", "ValidacionVeterinaria", conservando la correspondencia con los requisitos previamente especificados.

Dentro de este escenario, BOUML puede emplearse como herramienta CASE para transformar parte del diseño conceptual de MundiPets en artefactos de implementación. La herramienta permite representar clases, atributos, operaciones, visibilidades, asociaciones, generalizaciones y cardinalidades, elementos necesarios para definir la estructura estática del subsistema seleccionado. Un estudio desarrollado por Durai et al. utilizó BOUML para exportar diagramas UML mediante un esquema XMI, del cual se extrajeron clases, miembros, métodos y

llamadas necesarias para generar código automáticamente[5]. Este antecedente demuestra que los modelos creados con BOUML pueden actuar como una representación formal e intercambiable, capaz de alimentar procesos de generación de código y mantener una correspondencia explícita entre los elementos del modelo y los archivos resultantes.

Aplicado a MundiPets, el diagrama de clases desarrollado en BOUML podría emplearse para generar la estructura inicial de las entidades del dominio, incluyendo sus atributos, métodos y relaciones. Por ejemplo, una asociación entre Mascota e HistorialMedico debería reflejarse en el código mediante referencias o colecciones tipadas; de igual manera, las operaciones definidas en las clases podrían convertirse en firmas de métodos dentro del lenguaje objetivo. Este procedimiento permitiría disminuir la duplicación de trabajo, mantener convenciones estructurales uniformes y evidenciar la trazabilidad modelo-código exigida en la actividad. Además, el proceso de roundtrip controlado permitiría modificar un atributo, operación o cardinalidad en el modelo, regenerar el código y verificar que el cambio aparezca en el artefacto correspondiente, conforme a los criterios establecidos en la guía de evaluación.

La literatura reciente respalda el potencial de estas prácticas, pero también señala que la generación automática no elimina la necesidad de intervención técnica. Las herramientas basadas en modelos elevan el nivel de abstracción y pueden reducir parte del código escrito manualmente, aunque no todo desarrollo visual constituye necesariamente un proceso MDD completo[6]. Otros trabajos han demostrado que las transformaciones UML y modelo-a-texto pueden producir implementaciones coherentes y reducir errores repetitivos, siempre que existan reglas de transformación, estereotipos y mecanismos de validación claramente definidos[7],[8]. Una revisión sistemática reciente también identifica al paradigma modelo-a-código como una de las estrategias predominantes de generación automática, pero advierte sobre limitaciones relacionadas con la validación del código, el soporte multilenguaje y la necesidad de comprobar empíricamente los resultados obtenidos.

Por consiguiente, el uso de BOUML en MundiPets no pretende generar automáticamente la totalidad de la aplicación. Aspectos como las reglas de adopción y cruza, el control de acceso por roles, la protección de datos, la validación documental, la persistencia, las interfaces web y los componentes de inteligencia artificial requerirán configuración adicional o implementación manual. Su valor principal reside en convertir el modelo UML en una fuente estructurada para la generación inicial del código, facilitar la correspondencia entre requisitos, clases y artefactos de software, y permitir que los cambios del diseño puedan propagarse de forma controlada. En consecuencia, el objetivo del presente trabajo es demostrar la relación entre BOUML y el PFC MundiPets mediante el modelado de un subsistema representativo, la configuración del mecanismo de generación, la obtención y verificación del código fuente y la ejecución de un cambio controlado que evidencie la trazabilidad entre el modelo de origen y la implementación resultante.

Referencias

- [1] International Organization for Standardization and International Electrotechnical Commission and Institute of Electrical and Electronics Engineers, *Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, ISO/IEC/IEEE Std. ISO/IEC/IEEE 29148:2018, Nov. 2018. [Online]. Available: <https://www.iso.org/standard/72089.html>
- [2] Object Management Group, *MOF Model to Text Transformation Language, Version 1.0*, Object Management Group Std. formal/2008-01-16, Jan. 2008. [Online]. Available: <https://www.omg.org/spec/MOFM2T/1.0>
- [3] —, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, Object Management Group Std. formal/2017-12-05, Dec. 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1>
- [4] —, “Model driven architecture guide, revision 2.0,” Object Management Group, Tech. Rep. ormsc/2014-06-01, Jun. 2014. [Online]. Available: <https://www.omg.org/cgi-bin/doc?ormsc/2014-06-01>
- [5] A. D. Durai, M. Ganesh, R. M. Mathew, and D. K. Anguraj, “A novel approach with an extensive case study and experiment for automatic code generation from the XMI schema of UML models,” *The Journal of Supercomputing*, vol. 78, no. 6, pp. 7677–7699, 2022. [Online]. Available: <https://doi.org/10.1007/s11227-021-04164-x>
- [6] D. D. Ruscio, D. S. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?” *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022. [Online]. Available: <https://doi.org/10.1007/s10270-021-00970-2>
- [7] L. Jiménez-Navajas, R. Pérez-Castillo, and M. Piattini, “Code generation for classical-quantum software systems modeled in UML,” *Software and Systems Modeling*, vol. 24, no. 3, pp. 795–821, 2025. [Online]. Available: <https://doi.org/10.1007/s10270-024-01259-w>
- [8] L. Huning and E. Pulvermüller, “Automatic code generation of safety mechanisms in model-driven development,” *Electronics*, vol. 10, no. 24, p. 3150, 2021. [Online]. Available: <https://doi.org/10.3390/electronics10243150>